Universidad Politécnica de Atlautla

“Fundamentos de Inteligencia Artificial”

(Sección 5 de 6 – 2.5 Búsqueda con heurísticas)
(A * / Dijkstra)

Segundo Parcial

Docente: García Lara Javier

Equipo:
Sanchez Fragozo Dulce Maria - 24305056
Lopez Perez Veronica Juliette - 24305097
Jennifer Rivera Andrade - 24305072
Marin Garcia Danna Isla - 24305080

Carrera: Ingeniería en Tecnologías de la Información e Innovación Digital

Grupo: 171

25 de Octubre del 2026, Atlautla de Victoria Edomex

1. Planteamiento del problema

Una empresa de transporte necesita calcular la ruta más corta entre localidades del Valle de Chalco–Amecameca, la zona de Ozumba–Atlautla y el norte de Morelos. El objetivo es comparar dos algoritmos de ruteo —**Dijkstra** (sin heurística) y **A*** (con heurística)— en cuanto a calidad de la ruta, nodos expandidos y tiempo de cómputo, como ocurre en un sistema de rutas GPS (Russell & Norvig, 2020).

2. Formulación y datos

Componente	Definición
Estados	Localidad en la que se encuentra el vehículo (16 nodos)
Acciones	Tomar un tramo carretero hacia una localidad vecina (25 tramos)
Costo de paso	Kilómetros del tramo
Heurística h(n)	Distancia geodésica (fórmula de haversine) de n a la meta
Función de evaluación	Dijkstra: f(n) = g(n) · A*: f(n) = g(n) + h(n)

Aviso sobre los datos. Las coordenadas GPS de las localidades son aproximadas y el peso de cada tramo se estimó como distancia haversine × un factor de sinuosidad entre 1.15 y 1.45 (semilla fija = 7). No son kilometrajes oficiales de carretera; el código está preparado para sustituirlos por datos de OpenStreetMap o de una API de mapas.

3. Procedimiento

Ambos algoritmos comparten la misma implementación con cola de prioridad (heapq): Dijkstra usa h = 0 y A* usa la heurística haversine. Se evaluaron 6 pares origen-destino. El costo de cada ruta se validó contra *networkx.dijkstra_path_length* (Hagberg et al., 2008). También se probó en una red geométrica aleatoria de 1498 nodos (40 pares aleatorios, heurística euclidiana) y se calculó A* ponderado, f = g + w·h.

4. Resultados

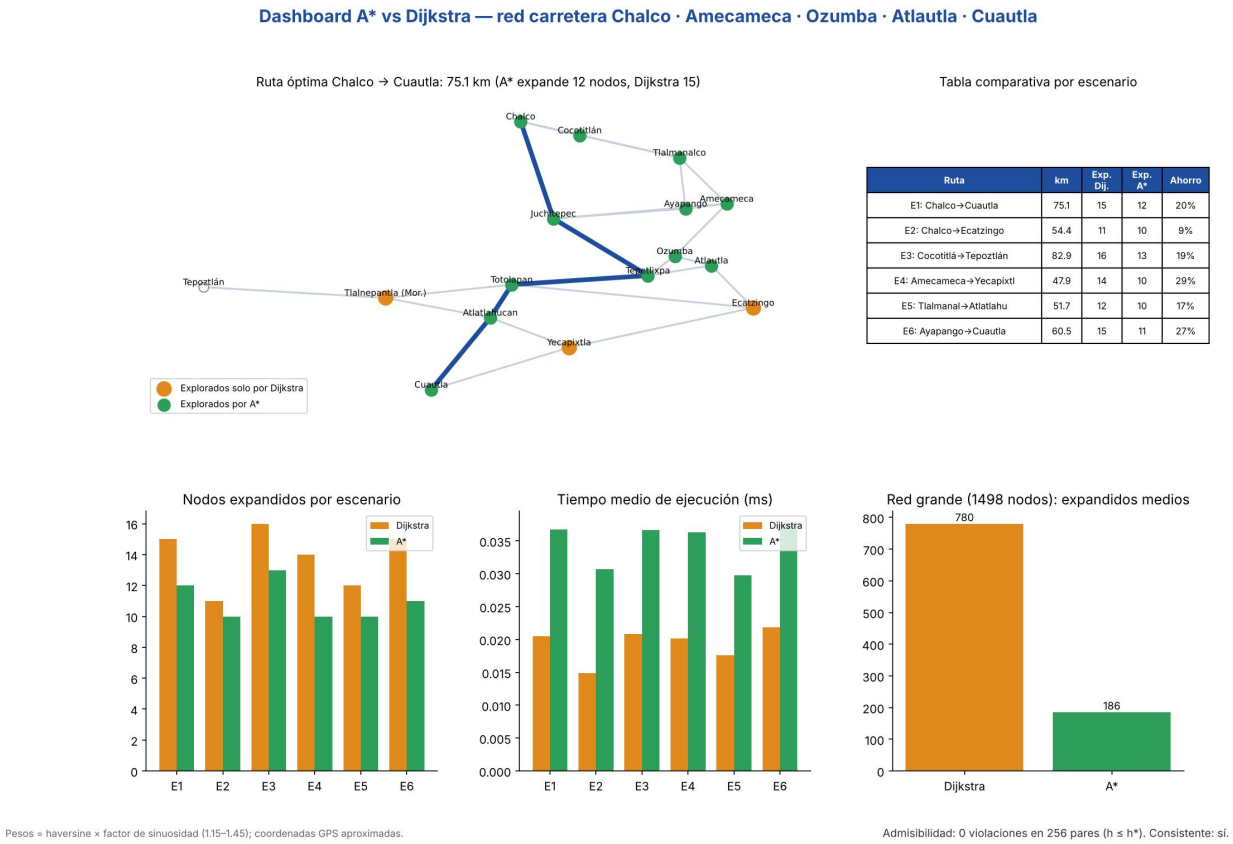


Figura 1. Dashboard: mapa de la red con la ruta Chalco → Cuautla, tabla comparativa, nodos expandidos, tiempos y prueba en red grande.

Escenario	Costo (km)	Exp. Dijkstra	Exp. A*	Ahorro	t Dijkstra (ms)	t A* (ms)
E1: Chalco → Cuautla	75.1	15	12	20 %	0.020	0.037
E2: Chalco → Ecatzingo	54.4	11	10	9 %	0.015	0.031
E3: Cocotitlán → Tepoztlán	82.9	16	13	19 %	0.021	0.037
E4: Amecameca → Yecapixtla	47.9	14	10	29 %	0.020	0.036
E5: Tlalmanalco → Atlatlahucan	51.7	12	10	17 %	0.018	0.030

Escenario	Costo (km)	Exp. Dijkstra	Exp. A*	Ahorro	t Dijkstra (ms)	t A* (ms)
E6: Ayapango → Cuautla	60.5	15	11	27 %	0.022	0.038

Rutas óptimas: la primera ruta (Chalco → Cuautla) es Chalco > Juchitepec > Tepetlixpa > Totolapan > Atlatlahucan > Cuautla.

Simulación animada

Al ejecutar python astar_dijkstra.py se abre una ventana con la búsqueda paso a paso: Dijkstra (izquierda) y A* (derecha) exploran la red y, cuando cada uno termina, un camión recorre la ruta encontrada con el contador de kilómetros. Con un clic se pausa y se reanuda. También se guarda como resultados_s5/simulacion_astar_dijkstra.gif. Se puede cambiar la ruta con --origen Amecameca --destino Cuautla. A* llega antes porque expande menos nodos.

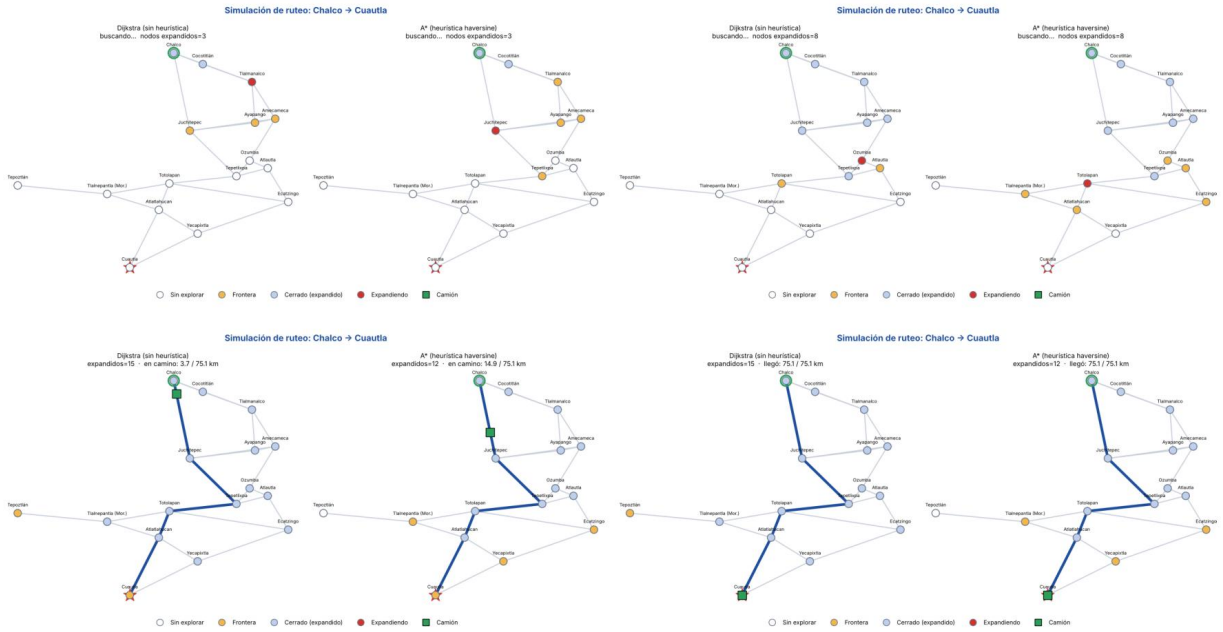


Figura 2. Fotogramas de la simulación: inicio, mitad de la búsqueda, ruta encontrada y llegada del camión.

5. Análisis de admisibilidad

Una heurística es **admisible** si nunca sobreestima el costo real al objetivo: h(n) ≤ h*(n). Aquí h(n) es la distancia en línea recta y cada tramo cuesta la línea recta × un factor ≥ 1, por lo que ninguna ruta real puede ser más corta que la línea recta. Se comprobó numéricamente en los 256 pares nodo-meta: **0 violaciones**, y la razón máxima h/h* fue 0.87. También cumple la **consistencia** (h(u) ≤ c(u,v) + h(v)) en todas las aristas, por lo que A* no necesita reabrir nodos y su resultado es óptimo.

Con A* ponderado (w > 1) la heurística deja de ser admisible y se pierde la garantía de optimalidad. En esta red pequeña, aun con w = 5 se obtuvo el mismo costo (75.1 km) y los nodos expandidos bajaron de 12 a 6; no es una garantía, sino un efecto de que la red tiene pocas rutas alternativas.

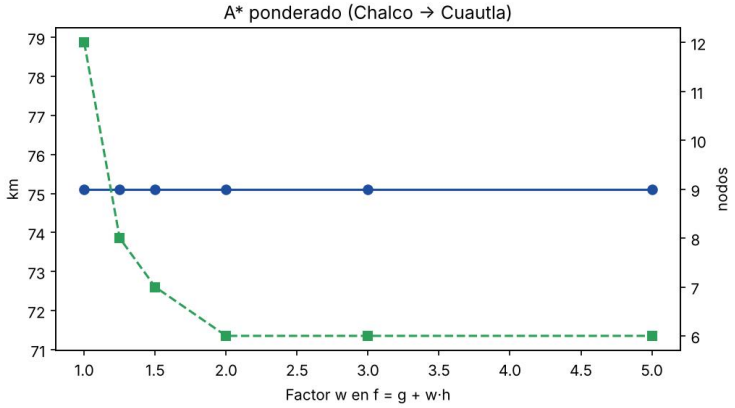


Figura 3. Efecto del factor w sobre el costo y los nodos expandidos (Chalco → Cuautla).

6. Conclusiones

- A* y Dijkstra encontraron rutas del mismo costo en los 6 escenarios, como se esperaba con una heurística admisible y consistente.
- A* expandió entre 9 % y 29 % menos nodos en la red de 16 nodos. En la red de 1498 nodos la reducción fue mucho mayor: 780 frente a 186 nodos en promedio (≈ 76 %).
- En la red pequeña A* fue *más lento* en tiempo (aprox. 0.03–0.06 ms frente a 0.02–0.03 ms), porque calcular la heurística tiene un costo que no se compensa cuando

ya se expanden pocos nodos. La ventaja de A* aparece al aumentar el tamaño de la red.

- Limitaciones: los datos son aproximados, la red es pequeña y no se modelan tráfico, sentidos de circulación ni tiempos de viaje. Como trabajo futuro, usar datos de OpenStreetMap y comparar contra tiempos reales.

7. Declaración de uso de inteligencia artificial

Elemento	Detalle
Herramienta	Claude (Anthropic, 2026), modelo Claude Sonnet 5.5 [modelo de lenguaje de gran tamaño]
Prompt	Se pidió ayuda para resolver la Actividad Parcial 2, Sección 5 (A* vs Dijkstra con dashboard) a partir de la hoja de instrucciones de la asignatura.
Uso	Generación del borrador del código Python, del dashboard, de la simulación animada, del README y de la estructura del reporte (Anthropic, 2026).
Verificación	Se ejecutó el código; los costos se validaron contra NetworkX, la admisibilidad se comprobó numéricamente y las cifras provienen de metricas.json. Las coordenadas y los pesos son aproximados y deben revisarse.

Referencias (APA 7.ª ed.)

Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Bohm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science.

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11–15).

Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Anexo. Código fuente y README

astar_dijkstra.py

```
"""
Sección 5 - Búsqueda con heurísticas: A* vs Dijkstra
Asignatura: Fundamentos de Inteligencia Artificial - UPA (Grupos 171 y 172)

Caso: ruteo en una red carretera regional (Valle de Chalco - Amecameca -
Ozumba - Atlautla - Morelos). Nodos = cabeceras municipales con coordenadas GPS
aproximadas; aristas = tramos carreteros; peso = distancia por carretera (km).

NOTA: los pesos se estiman como distancia geodésica (haversine) x factor de
sinuosidad (1.15 - 1.45). Para datos reales, reemplazar `construir_red()` por
una red descargada de OpenStreetMap (osmnx) o por una matriz de distancias de
una API de mapas; el resto del código no cambia.

Uso:  python astar_dijkstra.py
Genera: resultados_s5/dashboard.png, resultados_s5/tabla_comparativa.csv,
        resultados_s5/metricas.json y resultados_s5/simulacion_astar_dijkstra.gif

SIMULACIÓN: al ejecutar se abre una ventana con la búsqueda paso a paso (Dijkstra a la izquierda,
A* a la derecha) y un camión que recorre la ruta encontrada (clic sobre la ventana = pausar/reanudar).
Opciones:
  python astar_dijkstra.py --origen Amecameca --destino Cuautla
  python astar_dijkstra.py --sin-ventana        (solo guarda el GIF, útil sin pantalla)
  python astar_dijkstra.py --velocidad 2        (más rápido; 0.5 = más lento)
"""
import argparse
import csv
import heapq
import json
import math
import os
import time

import matplotlib
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter
import networkx as nx
import numpy as np

SALIDA = "resultados_s5"
os.makedirs(SALIDA, exist_ok=True)

# nombre: (latitud, longitud) aproximadas
NODOS = {
    "Chalco": (19.2634, -98.8975),
    "Cocotitlán": (19.2400, -98.8600),
    "Tlalmanalco": (19.2025, -98.7967),
    "Ayapango": (19.1167, -98.7928),
    "Amecameca": (19.1250, -98.7667),
    "Juchitepec": (19.1000, -98.8767),
    "Tepetlixpa": (19.0042, -98.8167),
    "Ozumba": (19.0369, -98.7997),
    "Atlautla": (19.0211, -98.7764),
    "Ecatzingo": (18.9500, -98.7500),
    "Totolapan": (18.9889, -98.9031),
    "Tlalnepantla (Mor.)": (18.9667, -98.9833),
    "Atlatlahucan": (18.9333, -98.9167),
    "Yecapixtla": (18.8833, -98.8667),
    "Cuautla": (18.8122, -98.9542),
    "Tepoztlán": (18.9847, -99.0986),
}

# tramos carreteros (grafo no dirigido)
TRAMOS = [
    ("Chalco", "Cocotitlán"), ("Chalco", "Tlalmanalco"), ("Cocotitlán", "Tlalmanalco"),
    ("Chalco", "Juchitepec"), ("Tlalmanalco", "Amecameca"), ("Tlalmanalco", "Ayapango"),
    ("Ayapango", "Amecameca"), ("Amecameca", "Ozumba"), ("Amecameca", "Juchitepec"),
    ("Ayapango", "Juchitepec"), ("Ozumba", "Atlautla"), ("Ozumba", "Tepetlixpa"),
    ("Juchitepec", "Tepetlixpa"), ("Atlautla", "Ecatzingo"), ("Tepetlixpa", "Totolapan"),
    ("Ecatzingo", "Yecapixtla"), ("Totolapan", "Tlalnepantla (Mor.)"),
    ("Totolapan", "Atlatlahucan"), ("Atlautla", "Tepetlixpa"),
    ("Atlatlahucan", "Yecapixtla"), ("Yecapixtla", "Cuautla"),
    ("Atlatlahucan", "Cuautla"), ("Tlalnepantla (Mor.)", "Tepoztlán"),
    ("Tlalnepantla (Mor.)", "Atlatlahucan"), ("Ecatzingo", "Totolapan"),
]


def haversine(a, b):
    """Distancia geodésica en km entre dos puntos (lat, lon)."""
    R = 6371.0088
    la1, lo1, la2, lo2 = map(math.radians, (*a, *b))
```

```
    d = math.sin((la2 - la1) / 2) ** 2 + math.cos(la1) * math.cos(la2) * math.sin((lo2 - lo1) / 2) ** 2
    return 2 * R * math.asin(math.sqrt(d))


def construir_red(semilla=7):
    rng = np.random.default_rng(semilla)
    G = nx.Graph()
    for n, (la, lo) in NODOS.items():
        G.add_node(n, pos=(lo, la), gps=(la, lo))
    for u, v in TRAMOS:
        factor = rng.uniform(1.15, 1.45)          # sinuosidad >= 1  => h admisible
        G.add_edge(u, v, weight=round(haversine(NODOS[u], NODOS[v]) * factor, 3))
    return G


def h_geo(G, n, meta):
    return haversine(G.nodes[n]["gps"], G.nodes[meta]["gps"])


def reconstruir(padres, meta):
    ruta, n = [], meta
    while n is not None:
        ruta.append(n)
        n = padres[n]
    return ruta[::-1]


def buscar(G, ini, meta, usar_h, peso_h=1.0, trace=False):
    """Implementación común: Dijkstra (h=0) y A* (h=heurística)."""
    t0 = time.perf_counter()
    g = {ini: 0.0}
    padres = {ini: None}
    h = (lambda n: peso_h * h_geo(G, n, meta)) if usar_h else (lambda n: 0.0)
    abierta = [(h(ini), 0, ini)]
    cerrados, cuenta, max_abierta = set(), 0, 1
    hist = []
    while abierta:
        max_abierta = max(max_abierta, len(abierta))
        _, _, u = heapq.heappop(abierta)
        if u in cerrados:
            continue
        cerrados.add(u)
        if u == meta:
            break
        for v, datos in G[u].items():
            nuevo = g[u] + datos["weight"]
            if v not in g or nuevo < g[v]:
                g[v] = nuevo
                padres[v] = u
                cuenta += 1
                heapq.heappush(abierta, (nuevo + h(v), cuenta, v))
        if trace:  # foto del estado de la búsqueda tras cada expansión
            hist.append(dict(actual=u, cerrados=set(cerrados),
                             abierta={n for _, _, n in abierta if n not in cerrados},
                             g=g[u]))
    dt = (time.perf_counter() - t0) * 1000
    ruta = reconstruir(padres, meta) if meta in g else []
    return dict(ruta=ruta, costo=g.get(meta, math.inf), expandidos=len(cerrados),
                cerrados=cerrados, max_abierta=max_abierta, tiempo_ms=dt, hist=hist)


def promedio_tiempo(G, ini, meta, usar_h, peso_h=1.0, rep=300):
    ts = [buscar(G, ini, meta, usar_h, peso_h)["tiempo_ms"] for _ in range(rep)]
    return float(np.mean(ts))


def verificar_admisibilidad(G):
    """h(n) <= costo real óptimo h*(n) para todos los n y cada meta."""
    peor, violaciones, total = 0.0, 0, 0
    consistente = True
    for meta in G.nodes:
        real = nx.single_source_dijkstra_path_length(G, meta, weight="weight")
        for n in G.nodes:
            total += 1
            hn = h_geo(G, n, meta)
            if hn > real[n] + 1e-9:
                violaciones += 1
            peor = max(peor, hn / real[n] if real[n] > 0 else 0)
    for u, v, d in G.edges(data=True):
        for meta in G.nodes:  # consistencia: h(u) <= c(u,v) + h(v)
            if h_geo(G, u, meta) > d["weight"] + h_geo(G, v, meta) + 1e-9:
                consistente = False
            if h_geo(G, v, meta) > d["weight"] + h_geo(G, u, meta) + 1e-9:
                consistente = False
    return dict(pares=total, violaciones=violaciones, razon_max_h_sobre_real=peor,
                consistente=consistente)
```

```
def escenarios():
    return [("Chalco", "Cuautla"), ("Chalco", "Ecatzingo"), ("Cocotitlán", "Tepoztlán"),
            ("Amecameca", "Yecapixtla"), ("Tlalmanalco", "Atlatlahucan"), ("Ayapango", "Cuautla")]


def red_grande(n=1500, semilla=3):
    """Grafo geométrico aleatorio para evaluar escalabilidad (tipo red vial)."""
    rng = np.random.default_rng(semilla)
    G = nx.random_geometric_graph(n, radius=math.sqrt(9 / (math.pi * n)), seed=int(semilla))
    G = G.subgraph(max(nx.connected_components(G), key=len)).copy()
    for u, v in G.edges:
        pu, pv = G.nodes[u]["pos"], G.nodes[v]["pos"]
        G[u][v]["weight"] = math.dist(pu, pv) * rng.uniform(1.0, 1.4)
    for n_ in G.nodes:
        G.nodes[n_]["gps"] = G.nodes[n_]["pos"]
    return G


def h_euclid(G, n, meta):
    return math.dist(G.nodes[n]["pos"], G.nodes[meta]["pos"])


def comparar_grande(G, pares=40, semilla=5):
    rng = np.random.default_rng(semilla)
    nodos = list(G.nodes)
    expD, expA = [], []
    for _ in range(pares):
        a, b = rng.choice(nodos, 2, replace=False)
        a, b = int(a), int(b)
        # Dijkstra
        rD = buscar(G, a, b, False)
        # A* con heurística euclidiana
        global h_geo
        original = h_geo
        h_geo = lambda GG, n, meta: h_euclid(GG, n, meta)
        rA = buscar(G, a, b, True)
        h_geo = original
        assert abs(rD["costo"] - rA["costo"]) < 1e-9
        expD.append(rD["expandidos"]); expA.append(rA["expandidos"])
    return float(np.mean(expD)), float(np.mean(expA)), len(G)


# ------------------------------ Dashboard ---------------------------------
AZUL, VERDE, GRIS, ROJO, NARANJA = "#1F4E9E", "#2E9E5B", "#C9D0DC", "#D1342F", "#E08A1E"


def dashboard(G, filas, admis, grande, ini_dem, meta_dem):
    fig = plt.figure(figsize=(16, 10.5))
    gs = fig.add_gridspec(2, 3, height_ratios=[1.25, 1], hspace=0.32, wspace=0.28)
    fig.suptitle("Dashboard A* vs Dijkstra — red carretera Chalco · Amecameca · Ozumba · Atlautla · Cuautla",
                 fontsize=15, fontweight="bold", color=AZUL)

    # (1) Mapa con rutas y nodos explorados
    ax = fig.add_subplot(gs[0, 0:2])
    pos = nx.get_node_attributes(G, "pos")
    rD = buscar(G, ini_dem, meta_dem, False)
    rA = buscar(G, ini_dem, meta_dem, True)
    nx.draw_networkx_edges(G, pos, ax=ax, edge_color=GRIS, width=1.6)
    nx.draw_networkx_nodes(G, pos, ax=ax, node_size=70, node_color="#FFFFFF", edgecolors="#8A93A6")
    nx.draw_networkx_nodes(G, pos, nodelist=list(rD["cerrados"] - rA["cerrados"]), ax=ax,
                           node_size=150, node_color=NARANJA, label="Explorados solo por Dijkstra")
    nx.draw_networkx_nodes(G, pos, nodelist=list(rA["cerrados"]), ax=ax, node_size=110,
                           node_color=VERDE, label="Explorados por A*")
    ruta = rA["ruta"]
    nx.draw_networkx_edges(G, pos, edgelist=list(zip(ruta, ruta[1:])), ax=ax, edge_color=AZUL, width=4)
    nx.draw_networkx_labels(G, pos, ax=ax, font_size=7.5, verticalalignment="bottom")
    ax.set_title(f"Ruta óptima {ini_dem} → {meta_dem}: {rA['costo']:.1f} km "
                 f"(A* expande {rA['expandidos']} nodos, Dijkstra {rD['expandidos']})", fontsize=11)
    ax.legend(loc="lower left", fontsize=8)
    ax.axis("off")

    # (2) Tabla
    ax = fig.add_subplot(gs[0, 2]); ax.axis("off")
    cab = ["Ruta", "km", "Exp.\nDij.", "Exp.\nA*", "Ahorro"]
    celdas = [[f"E{i + 1}: {f['origen'][:9]}→{f['destino'][:9]}", f"{f['costo']:.1f}", f["exp_dij"],
               f["exp_a"], f"{100 * (1 - f['exp_a'] / f['exp_dij']):.0f}%"] for i, f in enumerate(filas)]
    tabla = ax.table(cellText=celdas, colLabels=cab, loc="center", cellLoc="center",
                     colWidths=[0.52, 0.12, 0.12, 0.12, 0.14])
    tabla.auto_set_font_size(False); tabla.set_fontsize(8); tabla.scale(1.15, 1.8)
    for (r, c), cell in tabla.get_celld().items():
        if r == 0:
            cell.set_facecolor(AZUL); cell.set_text_props(color="white", fontweight="bold")
    ax.set_title("Tabla comparativa por escenario", fontsize=11)
```

```
    # (3) Barras nodos expandidos
    ax = fig.add_subplot(gs[1, 0])
    x = np.arange(len(filas)); w = 0.38
    ax.bar(x - w / 2, [f["exp_dij"] for f in filas], w, color=NARANJA, label="Dijkstra")
    ax.bar(x + w / 2, [f["exp_a"] for f in filas], w, color=VERDE, label="A*")
    ax.set_xticks(x); ax.set_xticklabels([f"E{i + 1}" for i in x])
    ax.set_title("Nodos expandidos por escenario"); ax.legend(fontsize=8)
    ax.spines[["top", "right"]].set_visible(False)

    # (4) Tiempo
    ax = fig.add_subplot(gs[1, 1])
    ax.bar(x - w / 2, [f["t_dij"] for f in filas], w, color=NARANJA, label="Dijkstra")
    ax.bar(x + w / 2, [f["t_a"] for f in filas], w, color=VERDE, label="A*")
    ax.set_xticks(x); ax.set_xticklabels([f"E{i + 1}" for i in x])
    ax.set_title("Tiempo medio de ejecución (ms)"); ax.legend(fontsize=8)
    ax.spines[["top", "right"]].set_visible(False)

    # (5) Escalabilidad + admisibilidad
    ax = fig.add_subplot(gs[1, 2])
    ax.bar(["Dijkstra", "A*"], [grande[0], grande[1]], color=[NARANJA, VERDE])
    ax.set_title(f"Red grande ({grande[2]} nodos): expandidos medios")
    for i, v in enumerate([grande[0], grande[1]]):
        ax.text(i, v, f"{v:.0f}", ha="center", va="bottom", fontsize=9)
    ax.spines[["top", "right"]].set_visible(False)
    fig.text(0.675, 0.035,
             f"Admisibilidad: {admis['violaciones']} violaciones en {admis['pares']} pares (h ≤ h*). "
             f"Consistente: {'sí' if admis['consistente'] else 'no'}.",
             fontsize=9.5, color="#333")
    fig.text(0.02, 0.035, "Pesos = haversine × factor de sinuosidad (1.15–1.45); coordenadas GPS aproximadas.",
             fontsize=8.5, color="#666")
    fig.savefig(f"{SALIDA}/dashboard.png", dpi=150, bbox_inches="tight")
    plt.close(fig)


def grafica_peso_heuristica(G, ini, meta):
    """A* ponderado: h inflada deja de ser admisible y puede perder optimalidad."""
    ws = [1.0, 1.25, 1.5, 2.0, 3.0, 5.0]
    costos, exp = [], []
    for w in ws:
        r = buscar(G, ini, meta, True, peso_h=w)
        costos.append(r["costo"]); exp.append(r["expandidos"])
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(ws, costos, "o-", color=AZUL, label="Costo de la ruta (km)")
    ax.set_xlabel("Factor w en f = g + w·h"); ax.set_ylabel("km")
    ax2 = ax.twinx(); ax2.plot(ws, exp, "s--", color=VERDE, label="Nodos expandidos")
    ax2.set_ylabel("nodos")
    ax.set_title(f"A* ponderado ({ini} → {meta})")
    fig.tight_layout(); fig.savefig(f"{SALIDA}/peso_heuristica.png", dpi=150); plt.close(fig)
    return dict(w=ws, costo=costos, expandidos=exp)


# ------------------------------ Simulación --------------------------------
def simular(G, ini, meta, velocidad=1.0, mostrar=True, archivo_gif=None):
    """Animación: Dijkstra (izq.) y A* (der.) buscan la misma ruta; cuando cada uno termina,
    un camión recorre el camino encontrado. Se ve cómo A* llega antes porque explora menos nodos.
    Blanco = sin explorar · amarillo = frontera (lista abierta) · azul claro = cerrado (ya expandido)
    · rojo = nodo que se expande ahora."""
    rD = buscar(G, ini, meta, False, trace=True)
    rA = buscar(G, ini, meta, True, trace=True)
    res = [("Dijkstra (sin heurística)", rD), ("A* (heurística haversine)", rA)]
    nodos = list(G.nodes)
    idx = {n: i for i, n in enumerate(nodos)}
    pos = nx.get_node_attributes(G, "pos")
    xy = np.array([pos[n] for n in nodos])
    pasos_tramo = 6
    max_busq = max(len(rD["hist"]), len(rA["hist"]))
    max_viaje = pasos_tramo * (max(len(rD["ruta"]), len(rA["ruta"])) - 1)
    total = max_busq + max_viaje + 14

    fig, axs = plt.subplots(1, 2, figsize=(13, 6.4))
    fig.suptitle(f"Simulación de ruteo: {ini} → {meta}", fontsize=13, fontweight="bold", color=AZUL)
    scs, camiones, rutas, textos = [], [], [], []
    for ax, (nombre, r) in zip(axs, res):
        for u, v in G.edges:
            ax.plot([pos[u][0], pos[v][0]], [pos[u][1], pos[v][1]], color="#C9D0DC", lw=1.5, zorder=1)
        sc = ax.scatter(xy[:, 0], xy[:, 1], s=130, c=["#FFFFFF"] * len(nodos), edgecolors="#6B7488",
                        linewidths=1.2, zorder=3)
        for n in nodos:
            ax.text(pos[n][0], pos[n][1] + 0.012, n, fontsize=6.8, ha="center", va="bottom", zorder=5)
        ruta_ln, = ax.plot([], [], color=AZUL, lw=4, zorder=2)
        cam, = ax.plot([], [], marker="s", color=VERDE, markersize=13, markeredgecolor="black", zorder=6)
        ax.scatter(*pos[ini], s=260, facecolors="none", edgecolors=VERDE, linewidths=2.5, zorder=4)
        ax.scatter(*pos[meta], s=420, marker="*", c="#D1342F", zorder=2)
        ax.axis("off")
        textos.append(ax.set_title("", fontsize=10))
        scs.append(sc); camiones.append(cam); rutas.append(ruta_ln)
    leyenda = [plt.Line2D([0], [0], marker="o", color="w", markerfacecolor=c, markeredgecolor="#6B7488",
                          markersize=10) for c in ("#FFFFFF", "#F2B84B", "#BFD0EE", "#D1342F")]
```

```
    fig.legend(leyenda + [plt.Line2D([0], [0], marker="s", color="w", markerfacecolor=VERDE,
                                     markeredgecolor="black", markersize=10)],
               ["Sin explorar", "Frontera", "Cerrado (expandido)", "Expandiendo", "Camión"],
               loc="lower center", ncol=5, fontsize=9, frameon=False)
    fig.tight_layout(rect=(0, 0.06, 1, 0.94))

    def cuadro(f):
        for i, (nombre, r) in enumerate(res):
            n = len(r["hist"])
            colores = ["#FFFFFF"] * len(nodos)
            if f < n:
                h = r["hist"][f]
                for c in h["cerrados"]:
                    colores[idx[c]] = "#BFD0EE"
                for a in h["abierta"]:
                    colores[idx[a]] = "#F2B84B"
                colores[idx[h["actual"]]] = "#D1342F"
                rutas[i].set_data([], []); camiones[i].set_data([], [])
                textos[i].set_text(f"{nombre}\nbuscando…  nodos expandidos={len(h['cerrados'])}")
            else:
                h = r["hist"][-1]
                for c in h["cerrados"]:
                    colores[idx[c]] = "#BFD0EE"
                for a in h["abierta"]:
                    colores[idx[a]] = "#F2B84B"
                ruta = r["ruta"]
                xs = [pos[x][0] for x in ruta]; ys = [pos[x][1] for x in ruta]
                rutas[i].set_data(xs, ys)
                t = min(f - n, pasos_tramo * (len(ruta) - 1))
                seg, frac = divmod(t, pasos_tramo)
                if seg >= len(ruta) - 1:
                    seg, frac = len(ruta) - 2, pasos_tramo
                fr = frac / pasos_tramo
                cx = xs[seg] + (xs[seg + 1] - xs[seg]) * fr
                cy = ys[seg] + (ys[seg + 1] - ys[seg]) * fr
                camiones[i].set_data([cx], [cy])
                km = sum(G[ruta[j]][ruta[j + 1]]["weight"] for j in range(seg)) + \
                     G[ruta[seg]][ruta[seg + 1]]["weight"] * fr
                llego = t >= pasos_tramo * (len(ruta) - 1)
                textos[i].set_text(f"{nombre}\nexpandidos={len(r['cerrados'])}  .  "
                                   f"{'llegó' if llego else 'en camino'}: {km:.1f} / {r['costo']:.1f} km")
            scs[i].set_facecolor(colores)
        return scs + camiones + rutas + textos

    anim = FuncAnimation(fig, cuadro, frames=total, interval=260 / velocidad, blit=False, repeat=True)
    estado = {"pausa": False}

    def al_clic(_):
        anim.resume() if estado["pausa"] else anim.pause()
        estado["pausa"] = not estado["pausa"]

    fig.canvas.mpl_connect("button_press_event", al_clic)
    if archivo_gif:
        anim.save(archivo_gif, writer=PillowWriter(fps=max(1, round(4 * velocidad))), dpi=80)
    marcas = [2, max_busq // 2, max_busq + 1, total - 3]
    fig2, axs2 = plt.subplots(2, 2, figsize=(12, 6.8))
    for ax2, f in zip(axs2.ravel(), marcas):
        cuadro(f)
        fig.canvas.draw()
        foto = np.asarray(fig.canvas.buffer_rgba())[..., :3]
        nb = np.where(foto.min(axis=2) < 250)
        ax2.imshow(foto[nb[0].min():nb[0].max() + 1, nb[1].min():nb[1].max() + 1])
        ax2.axis("off")
    fig2.tight_layout()
    fig2.savefig(f"{SALIDA}/fig_fotogramas_simulacion.png", dpi=150)
    plt.close(fig2)
    if mostrar and plt.get_backend().lower() not in ("agg", "pdf", "svg", "cairo", "ps", "template"):
        plt.show()
    plt.close(fig)
    return anim


def main():
    ap = argparse.ArgumentParser(description="A* vs Dijkstra con simulación animada")
    ap.add_argument("--origen", default="Chalco", choices=list(NODOS))
    ap.add_argument("--destino", default="Cuautla", choices=list(NODOS))
    ap.add_argument("--sin-ventana", action="store_true", help="no abrir la ventana; solo guardar el GIF")
    ap.add_argument("--velocidad", type=float, default=1.0, help="velocidad de la animación (1 = normal)")
    args = ap.parse_args()
    G = construir_red()
    assert nx.is_connected(G)
    filas = []
    for ini, meta in escenarios():
        rD, rA = buscar(G, ini, meta, False), buscar(G, ini, meta, True)
        ref = nx.dijkstra_path_length(G, ini, meta, weight="weight")  # verificación independiente
        assert abs(rD["costo"] - ref) < 1e-9 and abs(rA["costo"] - ref) < 1e-9
        filas.append(dict(origen=ini, destino=meta, costo=ref, ruta=" > ".join(rA["ruta"]),
                          exp_dij=rD["expandidos"], exp_a=rA["expandidos"],
                          t_dij=promedio_tiempo(G, ini, meta, False),
```

```
                        t_a=promedio_tiempo(G, ini, meta, True)))
    admis = verificar_admisibilidad(G)
    grande = comparar_grande(red_grande())
    pesos = grafica_peso_heuristica(G, "Chalco", "Cuautla")
    dashboard(G, filas, admis, grande, "Chalco", "Cuautla")
    simular(G, args.origen, args.destino, velocidad=args.velocidad, mostrar=not args.sin_ventana,
            archivo_gif=f"{SALIDA}/simulacion_astar_dijkstra.gif")

    with open(f"{SALIDA}/tabla_comparativa.csv", "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(filas[0].keys()))
        w.writeheader(); w.writerows(filas)
    with open(f"{SALIDA}/metricas.json", "w", encoding="utf-8") as f:
        json.dump(dict(escenarios=filas, admisibilidad=admis, red_grande=dict(
            dijkstra=grande[0], astar=grande[1], nodos=grande[2]), peso_heuristica=pesos),
            f, indent=2, ensure_ascii=False)

    print(f"{'Escenario':<32}{'km':>8}{'Dij':>6}{'A*':>5}{'t_D(ms)':>10}{'t_A(ms)':>10}")
    for fl in filas:
        print(f"{fl['origen'] + '→' + fl['destino']:<32}{fl['costo']:>8.2f}{fl['exp_dij']:>6}"
              f"{fl['exp_a']:>5}{fl['t_dij']:>10.3f}{fl['t_a']:>10.3f}")
    print("Admisibilidad:", admis)
    print("Red grande (Dij, A*, n):", grande)
    print("A* ponderado:", pesos)


if __name__ == "__main__":
    main()
```

README.md

````
# Actividad Parcial 2 · Sección 5 — Búsqueda con heurísticas (A* vs Dijkstra)

Fundamentos de Inteligencia Artificial · Universidad Politécnica de Atlautla · Grupos 171 y 172

Compara **Dijkstra** (búsqueda de costo uniforme, sin heurística) y **A\*** (heurística de distancia geodésica)
para encontrar la ruta más corta en una red carretera regional: Chalco · Amecameca · Ozumba · Atlautla · Morelos
(16 nodos, 25 tramos).

## Requisitos

- Python 3.10 o superior
- `pip install matplotlib networkx numpy`

## Cómo ejecutar (paso a paso)

1. Clona el repositorio y entra a la carpeta de la sección:
   ```bash
   git clone <URL-DE-TU-REPOSITORIO>
   cd <repositorio>/seccion5
   ```
2. (Opcional) Crea un entorno virtual: `python -m venv venv` y actívalo.
3. Instala dependencias: `pip install matplotlib networkx numpy`
4. Ejecuta: `python astar_dijkstra.py` (se abre la **simulación animada**; clic = pausar/reanudar)
5. Revisa la carpeta `resultados_s5/`:
   - `simulacion_astar_dijkstra.gif` — la animación guardada
   - `dashboard.png` — mapa de la red, tabla y gráficas comparativas
   - `tabla_comparativa.csv` — datos de los 6 escenarios
   - `metricas.json` — métricas completas (admisibilidad, red grande, A* ponderado)
   - `peso_heuristica.png` — efecto de inflar la heurística

## Simulación

La ventana muestra Dijkstra (izquierda) y A* (derecha) buscando la misma ruta, con colores por estado (blanco =
sin explorar, amarillo = frontera, azul claro = expandido, rojo = nodo actual) y un camión que recorre la ruta
encontrada.

```bash
python astar_dijkstra.py --origen Amecameca --destino Cuautla   # elegir la ruta
python astar_dijkstra.py --velocidad 2                          # más rápido
python astar_dijkstra.py --sin-ventana                          # solo guardar el GIF (sin pantalla)
```

## Cómo funciona

- **Estado:** localidad actual. **Acción:** tomar un tramo a una localidad vecina. **Costo:** kilómetros por
carretera.
- **Dijkstra:** expande por menor `g(n)` (costo acumulado).
- **A\*:** expande por menor `f(n) = g(n) + h(n)`, con `h(n)` = distancia haversine en línea recta a la meta.
- **Peso de cada tramo** = distancia haversine × factor de sinuosidad (1.15–1.45). Como el factor es ≥ 1, `h`
nunca sobreestima el costo real, así que es **admisible** (el código lo verifica en los 256 pares nodo-meta y
comprueba también la **consistencia**).
- Los resultados se validan contra `networkx.dijkstra_path_length`.

## Datos

Las coordenadas son aproximadas y los pesos son estimados (no son kilometrajes oficiales). Para usar datos reales,
sustituye `construir_red()` por una red de OpenStreetMap (`osmnx`) o por una matriz de distancias de una API de
mapas; el resto del código no cambia.

## Resultados principales

| Hallazgo | Valor |
|---|---|
| Rutas óptimas iguales en ambos algoritmos | 6 de 6 escenarios |
| Reducción de nodos expandidos con A* (red de 16 nodos) | 9 % – 29 % |
| Reducción en red de ~1 500 nodos (40 pares aleatorios) | 780 → 186 nodos (≈ 76 %) |
| Violaciones de admisibilidad | 0 de 256 |

En la red pequeña A* expande menos nodos, pero tarda un poco más por el costo de calcular la heurística; la
ventaja de tiempo aparece al crecer la red.

## Autores

Equipo — Grupo 171 · Docente: Mtro. Javier García Lara

## Declaración de uso de IA

Herramienta: Claude (Anthropic). Uso: apoyo en la estructura del código, generación del dashboard y redacción de
documentación. Verificación: el código se ejecutó, los resultados se validaron contra NetworkX y las cifras
reportadas provienen de la ejecución.
````